

4CS02 – Cryptologie

Travaux Pratiques

AES et RSA

Année 2025

1 Utilisation pratique d’AES

Dans cette première partie, nous allons considérer une exécution d’AES-128, et voir comme l’utiliser en pratique pour assurer la confidentialité, l’intégrité et l’authenticité pour un message de n’importe quelle taille.

Récupérez le fichier `4CS02_TP_AES.py` qui contient l’ensemble des opérations basiques d’AES (`SubBytes`, `ShiftRows`, `MixColumns`, `AddRoundKey`, ainsi que leurs versions inverses pour le déchiffrement), la génération de la clé étendue, ainsi que d’autres fonctions basiques qui pourront vous être utiles.

Gestion du padding. Pour un message de taille quelconque, il faut dans un premier temps pouvoir exécuter AES-128 sur des blocs pleins, c’est-à-dire 128 bits (i.e., 16 octets). Si ce n’est pas le cas du message que l’on cherche à protéger, il faut rajouter un *padding*. Il existe plusieurs façons de le faire mais nous allons ici utiliser celui décrit par le standard PKCS#7. Celui-ci fonctionne de la façon suivante : s’il manque N octets pour arriver à un multiple de 16, rajouter N fois l’octet N . Par exemple, s’il manque 4 octets, il faudra rajouter `0x04||0x04||0x04||0x04` à la fin du message initial.

Confidentialité, intégrité et authenticité. Comme vu en cours, nous allons utiliser les modes CBC pour l’intégrité et l’authenticité, et CTR pour la confidentialité, en utilisant le mécanisme d’*Encrypt-then-MAC*.

1. Implémentez un chiffrement AES complet avec un nombre de tours au choix, pour un bloc de 128 bits (16 octets). La génération des clés étant fournie, il vous reste donc à compléter le chiffrement et le déchiffrement en vous aidant de ce que nous avons vu en cours, et du code fourni.
2. Implémentez le padding PKCS#7, ainsi que le padding inverse, et intégrez-le au code précédent.
3. Implémentez le mode CBC que vous utiliserez avec AES pour calculer un MAC. Comment faut-il vérifier ce MAC ?
4. Implémentez le mode CTR que vous utiliserez avec AES pour chiffrer et déchiffrer un message.

5. Utiliser ces deux dernières étapes pour implémenter un *Encrypt-then-MAC* avec AES, le padding, et les modes CTR (pour le "encrypt") et CBC (pour le "MAC").

2 Implémentation de RSA-OAEP

Récupérez le fichier `4CS02_TP_RSA.py` qui contient un squelette de l'algorithme RSA et du padding OAEP.

1. Commencez par implémenter les différentes étapes de l'algorithme RSA (génération des clés, chiffrement et déchiffrement).
2. En vous aidant de ce qui est fourni dans le fichier, complétez votre code pour obtenir une version simplifiée du padding OAEP, et ainsi chiffrer et déchiffrer avec RSA-OAEP. Il faudra faire attention à la taille des éléments manipuler, afin que tout soit compatible. Le padding PKCS#7 peut aussi être utilisé pour cette étape. Voici par ailleurs quelques fonctions qui peuvent être utiles :
 - `get_hash_length` pour obtenir la taille de la sortie d'une fonction de hachage donnée en paramètre ;
 - `secrets.token_bytes` pour générer un aléa d'une taille donnée en entrée.

3 Principe d'encapsulation

Alice souhaite envoyer un message m à Bob, qu'elle souhaite protéger en confidentialité et en intégrité. Pour cela, elle va utiliser le mécanisme d'encapsulation. Ce dernier consiste dans un premier, pour Bob, à générer une paire de clés RSA. Nous considérons par la suite qu'Alice connaît la clé publique, et que cette dernière a été certifiée par une autorité compétente. Ensuite, Alice va procéder en plusieurs étapes : (i) générer deux clés AES aléatoirement (une pour la confidentialité et l'autre sur l'intégrité et l'authenticité), (ii) utiliser le système de chiffrement RSA pour chiffrer ces deux clés AES (voir partie 2), et (iii) utiliser les clés AES avec les modes CTR et CBC sur le message m (voir partie 1). De son côté, Bob devra déchiffrer les chiffrés RSA reçus pour récupérer les deux clés AES (voir partie 2), puis utiliser ses clés pour vérifier le MAC et retrouver le message m initial (voir partie 1).

1. En utilisant tout ce qui a été fait jusque-là, implémentez un tel mécanisme.